

Course Overview

Course Overview

Hi, everyone. My name is Henry Been, and welcome to my course, C# Dependency Injection, or for short, DI. I am a freelance DevOps and Azure architect from the Netherlands and have been working with .NET and C# for over 10 years. Understanding dependency injection and how to work with DI containers has been of great use to me in those years, and I hope to share along what I have learned. Some of the major topics that we will cover include what dependency injection is and how a dependency container helps write maintainable code, what lifetimes are and how you can use them to control the instantiation of your types, identifying a clear composition root in your applications, and further organizing dependency registrations in easy-to-reuse groups, many demos to explore design techniques like injecting groups of dependencies, the latest sensation, optional dependencies and interception. By the end of this course, you'll know enough about dependency injection in C# to help your team move to a more maintainable codebase. So I hope you will join me on this journey to learn about dependency injection with C#, here at Pluralsight.

Sandbox

Getting Started with a Product Importer

Introduction

Hello there, and welcome. My name is Henry Been, and it is my pleasure to be your teacher for this course on C# Dependency Injection. Dependency injection, often abbreviated as DI, is a well-known software development technique for writing large, yet maintainable solutions. The main thing we aim for when using DI is the decoupling of an interface or dependency from its actual implementation. But

before we get started with this course, let's take a look at the versions used throughout the course. This course was created using the displayed versions of C#, .NET, and Visual Studio. And this slide shows the versions of C#, .NET, and Visual Studio for which the information in this course applies. We will start this module by exploring why you need dependency injection. We will start by diving into a demo that will show you some of the inconveniences that you will run into when you are working without. We will then overcome them by adding a dependency injection container to our application. We will also take a look at some background and explore some alternative containers that are available for C# and touch upon some of the basics that all of them share. The module will complete by going over some theoretical background to ensure you not only know how to do the things we discuss, but also why. If you already have some experience with dependency injection in another language, you can choose to skip this module and advance directly to the next one. You can also continue with this module where we will dive straight into a demo now and see if we can get a feeling for why you need dependency injection.

Demo: Introducing Dependency Injection

In this first demo, you will get to know the application that we will be using throughout this course and expanding in the modules that follow this one. To do this, we will first browse the code and then complete it together. The application that we are going to work with is ProductImporter, a tool that is responsible for reading product information from a source and writing that to a target location. In this first iteration, we will read a CSV file, process it, and write the products found to another CSV file in a slightly different format. The sources of this application are available as part of this course as well. In the solution, you will find a file called product-input.csv that is an example of the type of files it can read. Let's also take some time to walk through the sources together so you can get a feel for how the application works. We start by opening the ProductImporter file. This is the class that contains the main algorithm and coordinates between reading and writing products. It has two references to other types that it needs for doing its work. It contains only one method that holds the actual algorithm. First, it opens both the source and the target, and then as long as there are products in the source, it gets them one by one and sends them to the target. Once all the products are sent over, both the source and the target are closed. The definition of a product source is captured in an interface called IProductSource. This interface defines the four methods you saw being called earlier. This interface is

implemented in a class called `ProductSource`. It also has a private field that holds a reference to `TextFieldParser`, a built-in type for reading delimited files like CSV, a reference to a class with two hard-coded constants, and a reference to a class that can parse the currency and amount out of a price. What follows are the methods for opening the source, which opens and configures the underlying `TextFieldParser` and the method for determining if there are more products to read. `Complexer` is the method for getting the next product. First, it gets the fields on the next line in the CSV file, and then it parses these fields into four local variables that hold specific information about the product. It uses these to instantiate a `Product` class and returns that as a result. Finally, there is the `Close` method that closes the underlying `TextFieldParser`. There is a similar setup for writing products. First, there is an `IProductTarget` interface, which is implemented in a class called the `ProductTarget`. This class also has a reference to the configuration. The `ProductTarget` has two private fields, one for holding a `StreamWriter`, which is used for writing the products to disk, and an `IProductFormatter` for formatting the products as strings that are valid lines in a CSV file. The CSV is formatted slightly different than the one that we are reading from. Instead of containing one field with currency and price, this field is now split into two, one with the currency and one with the amount. Here there are similar `Open` and `Close` methods, and an `AddProduct` method that takes the product, passes it to the formatter, and then writes the formatted result to file. Now let's revisit the `ProductImporter` one more time to review an important detail. Do you see how I not instantiate the `IProductSource` or `IProductTarget` in this constructor, even while the `ProductImporter` class clearly needs them? Instead, I add them as a parameter to the constructor, which indicates that I cannot construct a valid `ProductImporter` without supplying these types. I am declaring them as a dependency. This is the essence of dependency injection. Instead of constructing types you need, you get them injected through the constructor. But how do all these classes and interfaces work together? Now to hook everything up, we have to work in the `Program.cs` file. This file does not contain a class, but is the entry point for execution after compiling the application. So this is where I can instantiate the `ProductImporter` and then run it. And let's not forget to include the correct namespace. And this is where it gets tricky, as this code doesn't compile yet. As you can see, I also need to provide a `ProductSource` and a `ProductTarget` for instantiating the `ProductImporter`. So I have to instantiate these two types as well like this. And again, I need to include the correct namespaces. But again, I run into an issue, as they both have dependencies of their own. So first fixing the `ProductTarget`, I see it

needs a configuration and a productFormatter. So I instantiate those as well and pass them to the ProductTarget when instantiating it. Now on to the ProductSource. This also has two dependencies of its own, so I instantiate both of them and pass them to the ProductSource. Now here I make a mistake on purpose. Instead of reusing the Configuration class, I instantiate it again. Luckily, the compiler provides me with an error, but in larger solutions and with a different name, a mistake like this is easily made. Now in this case, I can fix it by removing the second instantiation, and the solution now compiles. Running it also works, as you can see from the result here. But going back to the code, you can hopefully see the problem that is being created here. Even for a simple solution like this, it is not easy and definitely not fun to manually track all the dependencies of all the classes and instantiate them manually and provide them where needed. Also, things like accidentally reusing instances when you shouldn't or not reusing them when you should is going to happen. And just imagine, you have hundreds of classes, and you have to figure out the order in which things to instantiate. What a nightmare. And on top of that, the file will become definitely unreadable over time when more classes are added. Especially when your solutions become very large with hundreds or thousands of classes, adding and maintaining a program file like this is an accident waiting to happen. Now please join me in the next clip to see how we can use a dependency injection container to get rid of these problems.

Checkpoint 01 - Applying Constructor Injection

Demo: Adding a DI Container

In this second demo, you will see how to add a dependency injection container to the ProductImporter for the first time. We will continue where we left off in the previous demo and redo the work we did to get there, but now using a dependency injection container. If you don't understand everything you see 100% right now, don't worry. In the remainder of this module, we will go over the background of what is happening in the code you see here. All right, let's go. To get started, we will first need to install a NuGet package from Microsoft. NuGet packages are the default .NET approach to reusing code written by others. To do this, we right-click the project and then choose Manage NuGet Packages. In the view that now opens, we can search in the Microsoft-hosted NuGet repository for the package we need, which is called Microsoft.Extensions.Hosting. This package contains the default approach for hosting applications, which includes the Microsoft.Extensions dependency injection package as a

dependency. As a default. I would use the Microsoft approach for hosting applications through this package. It doesn't just provide the DI container, but also all kinds of sensible defaults for, for example, configuration that we will be using later. Hitting Install installs the package into our project, and we can verify this by switching to the tab that lists all the installed packages. With the package installed, let's go back to our code in Program.cs and start using the application host and DI container we just installed. To make this possible, let's include the two new namespaces we will need to access the types that we are going to use. With this in place, we can now start using the application host by adding the following code. We call into a `Host.CreateDefaultBuilder` to start a default builder for the application host and then invoke `.Build`. This will build the actual host and store it in the variable called `host`. This application host can be used for setting up a number of things, including dependency injection. For adding DI, we will need to configure the builder by calling into another function called `ConfigureServices`. This function takes a lambda function as an argument, and that lambda function takes two parameters. The first one is the host builder, `context`, which we will not be using, and the second one is the service collection. The second one we will be using to register the types and classes we want to use. In the body of this lambda function, I can now add what are called my registrations. Here, I have to list all the classes I will be using so that the container can take instantiating classes over for me. The first class I register here is my `ProductImporter`. I used a reference to the service collection called `services` and invoke the method `AddTransient` and specify my class as a generic. This way, I make my class known to the container. Let's move the last part of my older code up to see how I change this to use the DI container. Instead of instantiating the `ProductImporter` myself, I now use the application host to get an instance. I do this by getting the services collection. This collection is built by the application host when we invoked a build method on line 13. Before line 13, the container is in the registration phase where we can add registrations like the one on line 11. But after building it, it is in the resolving phase where we can get instances from the container and then also using a different interface called `IServiceProvider`. On this interface, I invoked a method, `GetRequiredService`, and specify the type of the class that I need. The container will now look and see if there is a registration for that time, and if there is, it will instantiate it for me and return it to me. Once I've got the importer, I can leave the invocation of the `run` method untouched, but I'm not done yet. If I would run the code now, I would get an error stating that the `ProductImporter` cannot be instantiated because its dependencies are not known to the DI container. In other words, all

classes used by the ProductImporter have to be registered as well. So for a start, let's add our configuration class. Next, let's proceed with the PriceParser. And here you can see me doing something different. Instead of registering the type directly, I register both the interface and the class. This is because the ProductImporter declares a dependency on the interface, as you can see in its code. But at runtime, I want to work with the implementing type. Adding a registration this way, the container will return a PriceParser whenever an IPriceParser is requested. From here onwards, it is rinse and repeat and adding registrations for the other classes as well and cleaning up the old code. With all this in place, I can now run the program again, and it works as expected without any errors. Closing the console, at this point, it is important to observe a few differences with the code as it was before. First, I am no longer calling any constructor. Instead, the container will be doing that for me. Secondly, as a consequence, I no longer have to concern myself with ordering registrations, and I do not have to worry about the dependencies of each type. I just have to make sure that I register all the types I need to be available. This last thing makes these approach skills much better for a growing application, especially as it is possible to make these registrations modular, as you will see later on in this course. But for now, let's verify that the container is indeed doing what I claim it is doing. So I'm going to go into the ProductImporter class and add a breakpoint to the constructor. Next, I do the same thing for the ProductSource class. When I now run the application again, I expect to see the constructor of the ProductSource being hit, as it is here, and only then the constructor of the ProductImporter being hit, as it is here. So yes, the container is instantiating the classes for me, and it is also figuring out the order in which to instantiate classes. A great win. Now that you have seen some dependency injection and a DI container in action, please join me in the next clip to learn more about dependency injection containers.

Checkpoint 02 - Registering a dependency

Checkpoint 03 - Resolving a dependency

About Dependency Injection Containers

Now you have seen some dependency injection in action. Let's take a few minutes to learn a little bit more about DI containers. Dependency injection is a technique that you can realize manually, but in

practice, almost always, you will use an existing library. This is then called the DI container, and there are several DI containers available when you are working with C#. Autofac and Ninject are two of the most used containers for .NET that are not developed by Microsoft. Both are downloaded thousands of times each day by developers all over the world. Autofac has more daily downloads on NuGet than Ninject, and it appears to be used about five times as often. Both are good containers, and we will talk a bit more about them in one of the later modules. In solutions using the newer versions of .NET Core and .NET 5 or newer, you are less likely to encounter them than in .NET Framework solutions. A third container you might encounter in your work is the Unity Container. Unity is no longer under active development, and the corresponding GitHub repository has been archived. Originally, Unity was developed by Microsoft and handed over to the community in 2015. Up to that point, it was the most used DI container for C#. It's good to know the name, but you will not encounter it in newer solutions. In newer solutions, you will almost always encounter or introduce yourself the new default Microsoft DI container, Microsoft.Extensions.DependencyInjection. It is a built-in part of the .NET Framework and the most used DI container for newer solutions. It is also the container used in this course, except for when we will be talking about Autofac later. No matter which DI container you are using, they will share a lot of commonalities, and everything in this clip also applies to other containers than the new Microsoft.Extensions.DependencyInjection one. When working with a DI container, you will always work with it in one of two modes. At first, the container is in the registration phase. This is when you register your types, which you want to be able to resolve and the container should know about. At one point, you then build or complete the container, which makes it ready for the second phase, which is resolving types. When registering types, you are doing this so that the container knows about them and you can resolve them later. You cannot resolve types you have not registered beforehand. Most containers do not allow new registrations after building or completing the container configuration. Containers allow you to provide two types when making a registration. They are called the service type and the implementing type. The service type is the type that you want to make available for resolving, often an interface. The implementing type is the type that the container should instantiate and return. Sometimes these are the same, but sometimes they are not. In those cases, you are using polymorphism, and you can only use implementing types that inherit from or implement the service type. Finally, during registration, you can also specify a lifetime for the type. The lifetime determines when the DI container will instantiate a new service type or return an existing instance. As you might

recall from the previous demo, registering a type looks like this. First, you will need a variable with a reference to the service collection. Next, you call a method of which the name starts with Add. What follows is the lifetime like transient or singleton, and we will talk about lifetimes more in the next module. The method for registering a type takes two generic types, the service type and the implementing type. And when they are the same, you can omit the second one. The top line basically tells the container whenever somebody asks for an IProductSource, just give them a ProductSource. Once you have completed the configuration of the container this way, you can start resolving types. First of all, you can request types directly from the container, but the real power of DI becomes visible when you request a type that requires other types to be instantiated. In those cases, the container will first instantiate or find those dependencies and then use them to instantiate the class you requested. This also works for the dependencies of the dependencies of the class you request. The second responsibility of the DI container is to manage the lifetime of your classes. It will decide to instantiate a new type or use an existing type and dispose of types when it is sure you can no longer request them. In code, it looks like this. First, you have to get hold of a reference to the build container, in this case, the Services property of the application host. Once you have this, you can request any registered service type by calling the method, GetRequiredService, and specifying the type as a generic. The container will then create or return the requested type following the lifetime rules. Now you know how the DI container works in more detail. Let's talk next about two important principles in software development and how you can use the DI container to implement them.

Dependency Inversion and Inversion of Control

With a better understanding of how DI containers work, it is now time to spend a few minutes talking about two software design principles, dependency inversion and inversion of control. First up, dependency inversion, a principle that states that high-level modules should not depend on low-level modules. For example, your domain logic should not depend on how the code that connects to your database works. The following diagram shows an example of code where this principle is not applied. You see class A that to perform its work needs class B, which in turn needs class C. To relate this to the terminology of the principle, class A is a high-level module, class B, a lower-level module, and class C, an even lower-level module. All of these classes take a direct dependency on the class or classes they need. The consequence is that when class B changes, class A might have to change as

well. And the same goes for a change to class C, which might affect class B, which then in turn might affect class A. From experience, you might already know how this ripple of change can have a negative impact on the changeability of your solutions. A common approach to preventing this in object-oriented applications is the use of interfaces. Again, we have our class A, but this time it depends on interface B instead of directly on class B. Class B then implements this interface. And the same goes for the outgoing dependency of class B, which is on interface C implemented in class C. Now the most important thing here to observe is that the arrows are not in a long continuous line, but they confer it on the interfaces where the dependency line terminates. If these interfaces are designed well, this is also where any ripple of change would stop. If we now relate this principle back to the example application for this course, this means that we do not want direct dependencies from the ProductImporter on the ProductSource or from the ProductSource to the PriceParser. And if you recall the demos from earlier, you saw there that this was not the case. Instead, classes were decoupled through interfaces like this. The second principle is inversion of control, which states that your code should hand over control to a framework for determining which piece of code runs next. The following diagram shows a traditional flow where this is not the case. Here you see how we have a class A, which is the first class instantiated and called into at application startup. Class A then instantiates class B, calls code there, transferring control to B. B in turn instantiates class C and calls into that class. When inverting control, we are instead allowing the framework to decide which piece of code runs next. A concrete example of this is ASP.NET. Sometimes, even at application startup, the first piece of code that runs is cut from the framework and not our own code. The framework then decides which code to run first, for example, class A. Class A then needs some other code to run for specific action. But instead of locating and invoking the correct class itself, it requests an interface from the framework. The framework then determines which implementation to return, in this case, class B. And when class B needs yet another piece of functionality, it requests a cracked interface again from the framework. So the framework can decide to return class C and move control over to a method on class C. As you can see in this example, I have already applied the dependency inversion principle and added interactions through interfaces. Combining these two principles, you can build classes that are decoupled from other classes they use through interfaces and handing over control to a framework. This framework can be a DI container. Instead of having the framework run first, it is also possible to have your own code to resolve an initial type from the container and then rely on the

container to provide all dependencies. This is what we have done in the examples in this module. But as these principles might be a bit abstract at first, please join me in the next clip where we will go over more concrete recommendations that follow these principles.

Creating Maintainable Solutions Using Dependency Injection

This clip will give you some concrete pointers on designing your classes in a way that you can get the most out of dependency injection. First of all, all classes should have a single responsibility. You might already know this as the single responsibility principle. Following this principle and having classes expose only one or at most a few public methods helps to keep them focused on one thing and one thing only. When you take a dependency on another piece of code, do not depend directly on the implementing class, but instead depend on the abstraction that describes the capability you need, and do not think about which class might implement that capability and how. In C#, such an abstraction will often be an interface. Think of interfaces as being owned by the consumer, the class or classes that use them. This helps you to write interfaces that describe capabilities that are needed by a consumer of the capability and not interfaces that are just a reflection of the implementation. For example, I might need a capability to read files from disk. That is all I want to concern myself with and not how that is going to be implemented. When doing this, make no assumptions about the implementation. You should not have to go into a class to see how it works to be able to use it. Instead, knowing the interface that it exposes and the description of that interface should be enough to reliably use the implementation. If you do all of this, you are already implementing most of the concrete actions that follow from the dependency inversion and inversion of control principles. Just keep in mind that it is not enough to blindly follow these guidelines, but always keep thinking about why you do the things you do. Now let's recap what you have learned so far in the next clip.

Summary

Well done! You have completed this module. Let's recap what you have learned. First, we went through two demos to see the pain of manually providing dependencies to classes and how using a dependency injection container can help you remove that pain. Next, we took some time to go over some basics around dependency injection. We talked about different containers you can use and how

the new `Microsoft.Extensions.DependencyInjection` container has become the default option for most projects in C# and .NET. We talked about adding a container to your application, about registering types to make them available, and how we can resolve types once the container has been built. Finally, we went over some of the theoretical background and talked about dependency inversion and inversion of control and how they can help you to decouple your implementations from their interfaces, all that to create maintainable solutions. Now please join me in the next module where we will be taking a deep dive into the different lifetime options.

Lifetime Management

Introduction

Welcome back to C# Dependency Injection. In this module, we will be spending a significant amount of time talking about the different lifetimes that are available for you when working with the `Microsoft.Extensions.DependencyInjection` container. You will learn what lifetimes are, when to use which lifetime, and some tips to help you choosing. We will also talk about a very specific problem related to lifetime management called dependency captivity. And while we're at it, we will, of course, discuss how to prevent it. Now join me in the next clip where we will jump right into a demo talking about the singleton lifetime.

Demo: Reusing an Instance

In this demo, you will learn how to work with types that you want to instantiate only once. To practice with this, we will be adding a class that keeps importing statistics for our `ProductImporter`. And as with almost everything, I start by creating a new interface that will be the abstraction for this capability. I call this interface `IImportStatistics`. Once the file is created, I clean up the usings that we don't need and switch over to the file scope namespace declarations that come with C# 10 and up. I now paste in some code that I prepared for the interface and its implementation. The interface contains three methods, two for incrementing the count of products read and written and one for getting the results. Directly below is the class that implements this interface, which is quite to the point and, for example,

not thread-safe. If you want to follow along with this, you can also find this code in the assets that come with this module. I now use the Visual Studio hot keys for opening a menu with automatic refactoring for moving this class to its own file and automatically fixing the usings for the string builder class. Next, I open the ProductSource class to start sending statistics. I add the IImportStatistics interface as a dependency to the constructor and store the value provided in a private field. I then scroll down to the GetNextProduct method and increment a counter for products every time a product is read. And now move over to the ProductTarget class where I do something very similar. I add the interface as a dependency. And this time, I use the Visual Studio refactoring for storing the value provided. I go down to the AddProduct method, and here I also start gathering statistics, but now, of course, every time a product is written. Finally, I move over to the ProductImporter itself and inject our interface one more time and store a reference in a private field. I go down to the bottom of the algorithm and add one more line to write the statistics to the console. Earlier, you saw that not registering types leads to nasty exceptions. So let's be good developers and register this type with the container as well. I am adding a new entry as any other by using the method AddTransient. We have a service type of IImportStatistics and an implementing type of ImportStatistics. Now it is time to run the program and see the statistics being displayed on the screen. But, oh no. They're completely off. Everything is 0. How can this be? Well, the reason is that the container has instantiated a new ImportStatistics class three times, once for the ProductSource, once for the ProductTarget, and a third and final time for the ProductImporter. This means that the statistics are being written to the first two instances, but the statistics being displayed on screen are taken from the third instance where no statistics are recorded at all, but that's not what we need. We need all three of our classes to use the same instance to ensure that all statistics that are written are also displayed at the end. Luckily, this is possible. If I go back to the registration of my types, I can change the method call from AddTransient into AddSingleton and run the program again now getting the correct results. Changing our call to AddSingleton changes the lifetime of the type from transient into singleton, and singleton means that the same instance will be used whenever a dependency is requested. In the next clip, you will learn all about lifetimes, including a new third type of lifetime. I hope to see you there.

Checkpoint 04 - Registering a Singleton Dependency

About the Different Lifetimes

In the previous clip, you have seen the singleton lifetime in action and had a bit of a taste of how lifetimes influence your application. In this clip, we will go over some theory to better understand what you have seen. The most important takeaway from this module is that the lifetime you configure for types influences when the DI container will instantiate a new type or return a previously instantiated type. To influence this, you can choose from three different lifetimes. The first one is the transient lifetime, which was the lifetime we used when starting our work on the ProductImporter. When you define a transient lifetime for a type, the container will always instantiate a new class whenever an instance is required, no exceptions. Next up is the singleton lifetime. This lifetime you saw in the previous clip. Singleton is used when you want to always return the same instance whenever a type is requested. The container will instantiate the type the first time it is requested and keep returning the same instance as long as the container exists. Finally, there is the scoped lifetime. Here, the container will create one instance per scope. You will see scopes in action later, and we will talk about them a bit more then. But for now, it is easiest to understand the scope as a subcontainer. Scoped is the most difficult lifetime to understand, but don't worry. We will revisit all lifetimes using diagrams and code throughout this clip. Let's start by going over a diagram that explains how the transient lifetime works. Imagine we have a DI container and one scope created below that container. Again, think of this scope as a subcontainer. If we would now resolve a type with a transient lifetime on the container, we would get a first instance of it. Resolving it a second time provides a second instance, and resolving it on the scope will also provide a new instance every single time. Next up, singleton. Again, we have a container with one scope and start resolving a type, but this time, a type with a singleton lifetime. If we now resolve a type on that container, the container will create a first instance and return that. Resolving the type again will return the same instance, and the same goes for resolving a type on any scope under the container. The same instance will be returned every single time. Finally, there is the scoped lifetime. Here, you see a container with two separate scopes under it. If we would now resolve a type with a scoped lifetime on the container for the first time, it would create and return a type. The second time we resolve it, it would return the same instance. However, if we resolve it a third time, but now on the first scope, a new instance would be created again. A fourth resolve on the same scope will return that second instance again. But if the next resolve of the type is on another scope, we would get a third new instance. In other words, when using a scoped lifetime, the lifetime of the instance is

tied to the scope it created. And to complete that line of reasoning, you can think of the container as the root scope. Now let's look at these lifetimes one more time, but now using some code. And you can use these snippets yourself to get a feel for lifetimes if you want to practice on your own. First up, the transient lifetime. Here you see how you adjust your type with a transient lifetime using the `AddTransient` method. After building the container, it is now possible to resolve the service type twice, store those instances in different variables. And on the final line, I am using the `Object.ReferenceEquals` method to verify if those two variables hold the same instance or not. In this case, the contents of the final variable, our same instance, will be false. And this is what I expect when using a transient lifetime, as every resolve should yield a new instance. Now let's do the same for a singleton lifetime. First, the method used for registering is now called `AddSingleton`, as you might expect by now. Resolving two instances using the `GetService` method and storing them in variables, we can run the same test using `Object.ReferenceEquals`. This time, the variable, our same instance, would be true instead. Again, the expected result, as using a singleton lifetime will always return the same instance. Finally, the scoped lifetime. Here, the method for registering is called `AddScoped`. To see this in action, I am using a slightly different approach to the code. First, I am creating a scope from the container and store that scope in a variable called `firstScope`. And scopes are disposable, which is why I am using the `using` keyword. Now on the scope, I am yet again resolving the `ProductImporter` twice, and I compare the two instances again and expect them to be the same. Now I also create a second scope and resolve the same type a third and fourth time. This time I do two comparisons. First, I verify that the first instance, `AddScoped 1`, is not the same as the third instance, `AddScoped 2`. And finally, I confirm that the instances coming out of scope 2 are both the same again. Reproducing these examples yourself and playing around with them will greatly improve your understanding of lifetimes, and now you know the theory around lifetimes. Let's move into a demo where you can see the scoped lifetime in action, as that's the only one you haven't seen in the demo yet.

Demo: Transforming Products

In this demo, you will see how to work with the scoped lifetime. To prepare for this demo. I have made a few changes to the `ProductImporter`. You can find these changes in the assets that come with this course, but I will also walk you through the most important ones. The new requirement that we will be solving in this demo is that I want to apply a few transformations to products while importing them. I've

added an interface, `IProductTransformer`, that captures this capability and is used from the `ProductImporter`, as you can see here. We will be seeing more than one transformation, but a single interface describing this responsibility keeps the algorithm here nice and tidy. In the implementing class, we will have to solve the problem of applying the transformations one by one and counting the number of transformed products. To prepare further, I have also created two interfaces and implementing classes that realize specific transformations. They are the `ICurrencyNormalizer` interface and its implementing class. You can see it takes a `ProductTransformationContext` as a parameter, which it needs for getting the product at the start of the transformation and writing any updates. The same goes for the `INameDecapitaliser` and its implementing class, which you can see here. And I hope that from the name, the transformations that these types do, are clear enough. We have extended the `ImportStatistics` class to also accommodate for counting the number of products transformed. With this in place, it is time to start our work on the `ProductTransformer`. In this class, we have a dependency on the `IProductTransformationContext`, `INameDecapitaliser` and `ICurrencyNormalizer`. But we cannot inject them into the constructor because if we would inject them there, we would reuse the same instances throughout the lifetime of the `ProductTransformer`, which is instantiated only once. Instead, it is needed to tie the lifetime of these types to the product that we are transforming, so a new instance for every product. Luckily, we can use the scoped lifetime we talked about earlier to make this happen. To start using types with a scoped lifetime, the first thing we need to do is to create scopes, in this case, a scope per product. Creating scopes is done using the `IServiceScopeFactory`, which is a framework type that we can inject into our `ProductTransformer`. We saved a reference to a private field and are now ready to use it. I move into the `ApplyTransformations` method and invoke `_serviceScopeFactory.CreateScope`. The result I store in a variable called `scope`, and I also apply the `using` keyword, as it implements `IDisposable`. With the scope in place, I can access the service provider on the scope and invoke `GetRequiredService` to get a `ProductTransformationContext` on which I set the product we will be working on in this scope. Now before we proceed, a word of warning. Directly resolving classes on the container is called the service locator pattern and generally not recommended. In this case, we only do it to get more familiar with how the container works. And in a later module, you will see how to reduce direct dependencies on the container like this. All right, moving on, I can now resolve the two individual transformations in the same way and then execute the one after the other. After the transformations, I'm going to check with

the transformationContext if the product is changed by one or more transformations. And if it is, I want to increment the transformation count. To make this possible, I first have to go up to the constructor again and inject the IImportStatistics interface and save the reference to a private field. Now I can return to where I left off and increment the transformation count. And to complete the method, I return the resulting product back to the caller. Now for this to work correctly, it is important to ensure that all three types are instantiated exactly once for each product, especially the transformationContext cannot be transient, as it is reused by the ProductTransformer, NameDecapitaliser, and CurrencyNormalizer. And they all three need the same instance. It can't be singleton either, as it should not be reused between the transformations that are performed on different products. To make it so, we go to Program.cs where we register these three types and use the scoped lifetime, as mentioned before. Once this is done, it is also important to add a registration for the ProductTransformer itself, which can be transient, as there is no argument to give it another lifetime. If we now run the program, we see the desired outcome. Out of the three products that are in the input product CSV, two are transformed one or more times and one is not. If we open the input and output CSVs side by side, we can verify that this is indeed what has happened. This is a complete example that requires scoped lifetimes and shows you how to implement them. And now that you have seen all three lifetimes in action, let's meet in the next clip to investigate a problem that is specific to lifetimes called dependency captivity.

Checkpoint 05 - Registering a Scoped Dependency

Exercise 01 - Working with a ScopeFactory

Demo: Dependency Captivity

Lifetimes are a powerful tool that helps you declare when new types should be instantiated or when types should be reused. But as with any powerful tool, a small mistake can lead to grave errors. And in this clip, we will explore one possible mistake. We will be expanding the ProductImporter to generate a reference for a product every time it imports one. If there is ever an issue with a product in the future, we can then go back and look at what was imported and which line in the CSV it was from. To cater for this requirement, I have made a few changes to the code, and as always, you can find those in the

assets as well. First, I have updated the product model to accommodate for this new field called Reference. It is passed in through a new constructor and then saved to a public property. Secondly, I have added a new interface and called it the ReferenceAdder. This is a new type of transformation we will be applying to all our products, and it will be responsible for adding the reference to each product right here. To do this, it uses an IReferenceGenerator, which is implemented in this file. The ReferenceGenerator depends on the IDateTimeProvider for getting the current date and uses that to build a reference at this line. The second part of the reference is an ever-increasing counter. Finally, the DateTimeProvider. I have also prepared this class ahead of time, and it looks like this. At construction, it grabs the _currentDateTime, and it has a method for retrieving it later. It is now time to make the final changes together. We will open up ProductTransformer that is responsible for applying all the transformations and add the new transformation, the ReferenceAdder, in. We start by resolving this type, as we did with the other transformations. And once we have hold of an instance, we are going to execute that right after the other transformations. I'm also making one more change to slow the importing just that little bit down. This way, I ensure that we should be seeing different datetimes for each product reference. As always, I complete my work by registering the types with the container. So I go into Program.cs and add the three types we created. I use the scoped lifetime for the DateTimeProvider and ReferenceAdder, as they both relate to a single product being imported and thus to that scope. But I make the ReferenceGenerator a singleton, as it keeps that internal counter of products that I want to keep along for the full lifetime of the ProductImporter. It looks about right, but when I run the program and then grab some results, we see that things didn't work out as expected. For some reason, all products have a reference that starts with the same datetime. While we do see the counter, the last four digits, increasing, the datetime stays constant. Let's explore why that is. The problem here is that the DateTimeProvider is intended to be reinstantiated for every product, which is why it has a scoped lifetime. However, it is used by the ReferenceGenerator, which is a singleton. Now the fact that a ReferenceGenerator is instantiated only once makes that its dependencies are also gathered only once, in turn, making the DateTimeProvider effectively a singleton as well. This problem is called dependency captivity. The ReferenceGenerator is holding the DateTimeProvider captive, and it's a thing you want to prevent in your applications. Luckily, the container provides us with a way to automatically check for some type of dependency captivity. You can enable this by going into the Program.cs file and adding the following code. This code passes configuration options to the service

provider. And the option we are providing here is called `ValidateScopes`, which enables automatic detection of dependency captivity. If I now run the code again, I get a descriptive error stating what has gone wrong. One thing though, not all captivity errors are detected this way. For example, singleton and scoped types, depending on attention type, are not detected this way, as it is sometimes intended behavior and sometimes it's not. So here you have to be on your toes yourself. If you are wondering how to fix the problem of dependency captivity in this instance and want to think about that for a second, you should pause the video right now. All right, here are some solutions. A first solution would be to prevent the captivity by changing the lifetime for the `ReferenceGenerator` to `scoped` as well. To prevent this breaking the increment and counter, I would move this responsibility into a new type that can hold the singleton lifetime. In this setup, shorter lifetimes only depend on longer lifetimes, which fixes the problem. I will leave it as an exercise to you, the learner, to implement this in code. However, if you get stuck, you can always find a solution in the code that comes along with this course. Please join me in the next clip where we will go over some pointers to help you choose correct lifetimes and prevent lifetime errors as much as possible.

Checkpoint 06 - Enable scope validation

Which Lifetime to Select

As you might have concluded yourself by now, choosing the correct lifetime for your types is very important and can be challenging at times. In this clip, we will be going over some pointers to help you decide on lifetimes. The most important criteria for choosing a lifetime is how a type handles state and by state I mean private fields and properties on the class. For how long these values are valid determines the lifetime. Let's look at some pointers. If a class has no state at all, give it a `transient` lifetime. There is no reason to use a more complicated lifetime, and preventing unexpected behaviors here, for example, when the implementation of a class changes should take precedence. Even when the state is derived from some other class or is calculated on the fly, choose `transient` and do the recalculation when needed. Only when there was a problematic impact on performance, consider moving to a `scoped` or `singleton` lifetime, depending on what's appropriate. If the state in a class relates to a single item, request or context out of many available and needs to be shared between depending classes, choose `scoped`. A concrete example is a class in a web application that provides

information about the user that is currently logged in. Many frameworks, ASP.NET included, create a scope for a request that is handled by the framework. And on that scope, your request handling class is resolved. The logged-in user information is clearly related to a single request you are processing out of the many that come in. Thus, the containing type should have a scoped lifetime. If the state in a class relates to everything in the application to a singleton, an example can be the application configuration. In many cases, it is important that throughout the application, the same configuration is used. And giving the type that holds that configuration, a singleton lifetime makes sense. Finally, as a rule of thumb, try to stay away from instantiating custom scopes frequently. Custom scopes all over the place or worse, nested scopes, obfuscate how lifetime rules will be evaluated for your types. In an ideal world, you will never have to create scopes yourself, but can leave that to your framework. Again, ASP.NET is a great example. Now when putting this in practice, be careful with the decisions you make and test the consequences thoroughly. Incorrect lifetimes or incorrect expectations about the lifetime of classes can lead to unexpected behaviors or, as you might call them, bugs. When in doubt, go for the lifetime that is the easiest to understand and troubleshoot, the transient lifetime. Now if you are using third-party components or frameworks, take some time to learn what recommendations they give around lifetimes or which hard requirements they may pose. Often, they are not only good practices, but other developers that work in the same codebase may assume you follow the framework recommendations. One example is Entity Framework Core, which provides a class called `DbContext`. The `DbContext` should correspond to one database transaction or unit of work. In practice, it often means that the `DbContext` is injected with a scoped lifetime. Combining this with the ASP.NET scoped lifetime per request, this results in a one database transaction per request, often a desirable property. Another database that you might be working with is Azure Cosmos DB. The Cosmos DB SDK provides another class for interacting with the database called `CosmosClient`. As Cosmos DB does not support traditional transactions, it provides a single thread-safe client and comes with the recommendation to instantiate it as few times as possible, ideally just once, as this allows it to use as much internal optimizations as possible. In this case, a singleton lifetime for the `CosmosClient` makes sense. As you can see, there are a few hard rules, and choosing a lifetime becomes easier over time and with experience. The best way to get that experience is by trying things out, which I highly recommend. In the next clip, we will revisit some lifetime choices in the `ProductImporter` and update

the importer to write products to a database instead to get more feeling for the different lifetimes. See you there.

Demo: Choosing the Correct Lifetime

In the previous clip, we have gone over some pointers around lifetimes and which lifetime to choose when. In this demo, we will revisit the lifetime choices we made to see if there are any improvements and why I chose the lifetimes I chose. We will also add some code to write new products to a SQL database instead of to another CSV file. This will allow us to practice with the Entity Framework DbContext class. If you are not familiar with Entity Framework for accessing databases and you want to learn more about it, I recommend this course by Julie Lerman. Let's start our work by revisiting the choices made so far. And the choices that I made around lifetimes can be divided into two parts. The lower half contains registrations of types that are related to the transforming of a single product. Here, the scoped lifetime ensures that we keep the same instance of these types around while working with a single product, but don't keep them around for more than one. The only exception is the IncrementingCounter. This is to ensure that we have a global, ever-increasing counter, which also makes sense. The upper half of the types is more concerned with the importing in general. Here, I would suggest changing the lifetime of the configuration class to a singleton. While we will see other approaches to configuration later in this course, at this point, it is a good practice to guarantee that everyone who uses the configuration will get the same instance. For example, if there are ways to change the configuration during the runtime, this change ensures that the new settings would be available to everyone. The other classes here are transient, and they feel like they could be made singleton as well. And of course, you can test this in your own code and see that the program still functions correctly because they are all instantiated only once, and it might be tempting to make this change. However, it isn't necessary, and that is why I would recommend against it. If you would ever change the program to run multiple imports, you will probably want all types to be reinstantiated, thus I'm leaving this at transient. Let's also take this opportunity to look at another lifetime question now around Entity Framework Core. To prepare for this, I started by installing two more NuGet packages. Earlier, you saw how to install them through the UI, and this time, I installed them by adding two lines to the project file. Next, I create a class called TargetContext, which is the Entity Framework Core approach to defining a database connection. The context contains a DbSet, a collection of products,

which represents a table in the database. And you can define a model class, which describes the columns in that table and thus how entities should be stored. I will be using this class from a new `IProjectTarget` implementation. And to keep my code clean and understandable, I have first renamed the existing product target, the `CsvProductTarget`, which clears the way for the addition of this new class, `SqlProductTarget`. In this class, I reimplement the `Open` method to make a database connection and ensure the database and table schema are created. A specific close operation is not needed for the SQL target. In the `Add(product)` method, I use the Entity Framework Core approach to adding a product to the database. And please observe that in both cases, I am creating a new scope on which I resolve the `DbContext`. This allows me to register the context as a scoped lifetime, ensuring I have one database transaction per product written. To complete this change, I once more go into the `Program.cs` file and make two edits. First, I update the implementing type of `IProductTarget` to `SqlProductTarget`, and I have to add the registration for Entity Framework Core. Now I am not using one of the three methods you have seen before. Instead, I use the `AddDbContext` method, which is an extension method provided by the EF Core Framework to automatically add all the registrations for Entity Framework Core at once. And a spoiler alert here, you will learn how to use methods like this to better organize your own DI registrations later on. For now, it is enough to understand that this registers all Entity Framework Core types and passes along the connection string to MySQL DB, which is stored in the `appsettings.json`. And of course, I don't show the full string on purpose, as you should never expose passwords, even of those should invalidate directly after recording a demo. And with all these changes in place, I can now run the program once more. And when I do, you see that EF Core, by default, writes some logging to the console. Let's just ignore that and bring in my browser connection to the database instead. So see that at first there were no records here, but when I refreshed the page, there are, and that completes us practicing with lifetimes. Let's recap what you have learned in this module in the next clip.

Summary

And that completes this module about lifetime management. In this module, you have learned all about the three different lifetimes available, transient, scoped, and singleton. You have learned how they work internally and how to register and use them from your own code. We have also taken a look at dependency captivity where instances with a longer lifetime hold on to instances that are intended

to have a shorter lifetime, and you have learned how to detect and prevent it. Finally, we went over to sign recommendations and tips to help you choose the correct lifetime when registering classes. Now please join me in the next module where we will be expanding our use of the container to work with HttpClient and configuration and look at some utility methods.

Expanding the Product Importer

Introduction

Welcome back to C# Dependency Injection. In this module, we will be expanding our ProductImporter to practice with different possibilities that the default .NET DI container gives you. We will start by talking about ways to organize your solution and the DI registrations that come with it to create a more maintainable solution. Next, we will dive into two programming topics that you will often solve using the DI container. They are application configuration, using the options pattern, and the configuration and injection of the HttpClient. From there we will move on and discuss the service locator pattern, an approach where you do not inject dependencies, but the container itself. You will learn when you might have to do this, but also why it is better to avoid this. Finally, we will spend a few words on the dependency injection and how this helps with increasing testability. Now please join me in the next clip where we will talk about composition roots.

Demo: Separating out the Entrypoint

In this demo, we will talk about the composition root of an application. The composition root is the place in your application where you have an overview of all components involved and knit everything together. When you use a DI container, it is the place where you make your registrations. To help with our discussion of this topic, I have prepared some changes to the ProductImporter. This time, I have not made any changes to the code, but I have spread the existing code out over three new class libraries in a way that is similar to many solution architectures in the real world. At the bottom, you see

a models project that contains the class that our code operates on. The logic, I've split into two parts. The main algorithm is in the ProductImporter.Logic library, and all the code that contains and supports the specific transformations is in ProductImporter.Logic.Transformations. I've also updated the usings in Program.cs so everything still compiles and works. If we now inspect this file in more detail and look closely, we will see that there are two responsibilities in this file. At the top, we see how it registers all the types with the DI container. This part is the composition root, but that's not everything there is. Below this, we also see two more lines that actually resolve the first time and run the program. This code, together with the building of the application host, we call the entry point. It is the code that runs at startup. If we look at a diagram, it looks like this, a composition root and an entry point, two different concepts that are now kind of intertwined. Now imagine that we do not want to run our ProductImporter only from the command line, but also as part of an ASP.NET application. In that case, the startup code of that application would be a second entry point. Now instead of repeating all the registrations, we can separate the composition root out into a separate project. Our entry point will then depend on the composition root, which will in turn depend on all other projects and knit all DI configuration together. We can then reuse this composition root from other entry points we might add later. So let's make this change together. The composition root is a new project, so I go to my solution, right-click, and in the menu, select Add, New Project. There should be a class library, and the name will be ProductImporter.CompositionRoot. Once created, the first thing I do is remove the auto generated Class1. Next, I go into the Project file and add the Microsoft.Extensions.Hosting dependency here as well. Next up is the reordering of project dependencies. For my composition root, I take a dependency on all projects that contain my types. And for my entry point, I remove all the existing dependencies and add a dependency on only the composition root. Now it is time to add some code and add a new class to the composition root. Let's call it ProductImporterCompositionRoot. I clean up the file and make the class a public static class so I can add extension methods. I add an extension method called AddProductImporter, which is an extension to the IServiceCollection, which it also returns. I add the correct using, and I am now ready to add the body of the method. And the body is going to be the list of all the registrations that are now in Program.cs. So I go there and copy them out from there into my composition root. While we are at it, this is also a good time to explore the difference between Add and TryAdd. When you invoke AddScoped, AddTransient or AddSingleton, you are adding a registration if there isn't one yet. However, if there is one already, you will overwrite it.

This is not the case when you use the TryAdd equivalent. This method will only add a registration if there isn't one yet. If you are building reusable modules like the composition root we are creating now, you might want to choose TryAdd over Add in certain cases to prevent overwriting already registered times. Now with that in the back of our mind, let's go back to the code to wrap things up. To complete our introduction of a composition root, we have to go back to Program.cs where we add a call into our extension method so that the entry point now uses the composition root for registering all the types. And with that change made, I can now run the program and see that everything still works. With the introduction of a composition root, let's look at how we can organize our registration even further in the next clip. See you there.

Checkpoint 07 - Add a composition root

Demo: Organizing Registrations

In this demo, we will take another good look at our CompositionRoot and try to improve its organization by grouping DI registrations. For a start, let's open up our existing CompositionRoot and take a look. If we continue this approach to organizing our registrations, this will become a very long, hard-to-maintain list over time. We need a way to split this list into something more maintainable. A good approach for this is by creating extension methods on the IServiceCollection type that take care of a logic grouping of registrations, and that's what we're going to implement in this demo. We start by creating a new class in the ProductImporter.Logic project called DIRegistrations. We make this class public and static so it can be used for extension methods. The first method we're going to add is a public static method that returns an IServiceCollection and also takes one as an argument. I first satisfied a return by returning the same instance as was passed in. Returning the same instance makes this method usable in a fluent way. I now need the correct using and a NuGet package for the IServiceCollection. With this in mind, I now go into the existing CompositionRoot and copy all the registrations for the main logic out and copy them into this class. And this module is now ready to go. From here, it is rinse and repeat to create a second class called DIRegistrations in my ProductImporter.Logic.Transformations project. Again, I add an extension method, but this time I call it AddProductTransformations. I implement the fluent signature and install the correct NuGet package. I now go into the old CompositionRoot and copy the remaining registrations out. I copy the name of this

method, and I fed in the `CompositionRoot`. I also invoke the other extension method. And at this point, our program runs exactly as it did before, yet our code is now much more modular, and the registrations are better organized. The following diagram reiterates this point. Here you can see how we moved individual registrations out of the `CompositionRoot` into a class specifically added for this responsibility. You also saw how we put this class close to the type that it registers, often in the same library. We can repeat this multiple times. And if you work with NuGet packages, you will see that most of them use this approach as well. You have already seen this when adding the Entity Framework Core configuration in the previous module. One question that might now come up is what if I want to reuse these modules with registrations, but with a small variation? There is a well-known approach to this problem, and it works as follows. First, we add a small class that contains the configurable options that we want to define. For example, enabling the `CurrencyNormalizer` or not. Once this type is created, we instantiate it at the top of our list of registrations. We then assume a variable, `optionsModifier`, that takes this instance and potentially modifies it. And as we can't just assume variables, we will take this method as a parameter. Within the registrations, I can now make decisions based on the values in that object. For example, I either register the `CurrencyNormalizer` or an empty implementation of this interface when the `CurrencyNormalizer` is not enabled. Finally, to make everything compile again, I go into my `CompositionRoot` and provide then a mandatory parameter as a lambda function, either like this, or if you like the full syntax better, like this. And this completes our investigation into organizing and grouping registrations. So please join me in the next clip to see how we can manage application configuration using the DI container.

Demo: Application Configuration

In this demo, you will learn how to work with application configuration through the DI container. Instead of having a central application that all classes read their configuration from, we will switch to an approach where classes describe which configuration they need and leave it to the DI container to provide that configuration. To start, we will go into our solution and remove the configuration class and its dependency registration. As our product source cannot go without some configuration, we will need to implement this in another way. We start by creating a new class that describes the configuration that the product source and only the product source needs. I call this type `ProductSourceOptions` and provide it with one property for which I find and reuse the existing name on the now removed

configuration class. To keep the code base clean, I move this type to its own file. Now it is time to get this configuration into our class. We do this by injecting an `IOptions<ProductSourceOptions>`. With this interface, we clearly indicate that this are configuration options that the class requires, not some random dependency. I update the field name and make the same changes to the constructor and save the injected value to the private field. To use options, we will need to install another NuGet package. For this, I open my project file and copy paste the existing line and reference a new package called `Microsoft.Extensions.Options`. I save the file and rebuild the solution to ensure the type is now available. Once that's done, I can add the correct using. Finally, I can update the non-compiling reference to the now removed configuration by a reference to this options instance. I get its value and then again, my source CSV path. We repeat these changes for the CSV product target. We update the private field to use an `IOptions<CSVProductTargetOptions>` and update the constructor and the assignment of the field accordingly. Here, I can generate the correct usings directly as the package is now installed. I also update from using the configuration to using the value provided through the options pattern. This is all that's needed to consume options. We now have successfully built the existing configuration type out, and the solution compiles again. However, it will not run successfully until we also provide the container with some way for providing these option classes. So let's do that next. As a first step, I update my `appsettings.json` to now hold all the configuration for my application, the default location for settings in .NET applications. To get the values from there into the options objects, we will be using the configuration binder, which is a NuGet package we first have to install. I again add a new line to my CS profile and now add `Microsoft.Extensions.Configuration.Binder`. The dependency is automatically installed, and a few seconds later, the types are available. I go into my DI registrations to complete the configuration. I will use a method called `addOptions` and provide the type that contains the options as a generic parameter, `CSVProductTargetOptions` in this case. This returns an object on which I can chain a call into the `configure` method that takes zero or more generic parameters. These parameters specify types that the DI container needs to inject in the `labdev` function that is provided as a parameter. In this case, I need an `IConfiguration` instance, which is a service that is part of the framework and provides access to all the configuration, including configuration in `appsettings.json`. In the body of the `labdev` function, I use these instances to get a specific section of the configuration. If I quickly flip back to the configuration, you can see what I mean by a section. A section are all the keys where the part before the colon is the same. And this part for

the colon is then called the section name. The part after the colon is the property name. I use the configuration to get my section by name, and then I bind this section to the options object. This means that all object properties with a name that is also present in the configuration section will get the value that is specified in the configuration automatically set. I skip the section name for a second, as I really don't like hard coding values here. Instead, I generate a new constant on the CSV product target options called section name and provide a name for the section here. Once again, I repeat all my actions, but now for the configuration of the product source. I update the type of the options object in both locations, and I also make a new section name property. To prove everything works, I run the application. And voila, you can see for yourself, everything still works. Now you know how to inject configuration into your applications using the options pattern, a default and recommended approach for using C# and .NET. Let's move on to the next clip, where you will see how to inject multiple implementations of the same type.

Checkpoint 08 - Register an Options type

Demo: Multiple Implementing Types

There are many situations where you need to inject not just a single dependency, but a group of dependencies. In our example, this is the case when we are transforming products. In this demo, you will see how you can inject an `IEnumerable<T>` to get multiple implementing types at once. We start our work in the `ProductTransformer` class. Here, we removed three individual resolves for the transformation and replace it with a single resolve of `IEnumerable<IProductTransformation>`, a new interface we will add to our codebase shortly. Now that we have a list of `IProductTransformers`, instead of having three separate ones, we can now replace the three implications of `Execute` and replace it with a loop over the transformers and invoke them one by one from the body of the loop. This change has made our code much more flexible. It does not have to resolve every individual type, which we want to afford as much as possible, is at the service location, which we will discuss later. We are also more flexible, as we can now work with 2, 20 or even 0 product transformations without ever changing this code. To continue making this work, we have to add this new interface we just defined. I clean up the file a bit and then give it an `Execute` method. We can now go over the four classes that first had an individual interface and replace that with the one we just created. We then remove the three interfaces

we don't need any more. If we now recompile, we see that everything compiles, except for the registrations, so let's update those as well. The one update I have to make here is the registration itself. So I register an implementing type under the same service type three times. The consequences of this are if I request an `IProductTransformation` directly, I will get the implementing type that is registered last. If I request an `IEnumerable<IProductTransformation>`, I will get an enumerable with all the implementing types. Let's prove that the latter works by going back to our `ProductTransformer` and setting a breakpoint. If I now run the application again and inspect the value for `ProductTransformers`, you will see that all three implementing types have been injected just as we wanted. Now before we wrap up, let's quickly circle back to what happens if I now resolve a single instance of `IProductTransformation`. In that case, I will only get the latest registrations. In other words, it feels like registrations overwrite each other. If you want to prevent this behavior, you can use the methods `TryAddSingleton`, `TryAddScoped`, and `TryAddTransient` to ensure you don't overwrite an existing registration. I could prove this by going back to my registrations and use a `TryAddScoped` method for one of the `IProductTransformation` implementations and run the program again. We will now see that there are only two transformations returned. The third one is not added due to the use of the `TryAdd` method. A prime example for when you might be using this approach is when you want to register a default, a registration that should only be done if there is no other registration yet. Now let's switch gears a bit and look at how we can inject an `HttpClient` into our application in the next clip.

Demo: Injecting HttpClient

Just like configuration, the use of `HttpClient` is something you can configure using the DI container. In this demo, we will explore that capability by reading our products from an internet address instead of from a local file. To prepare for this change, I have made a few changes to the codebase. Doing HTTP requests counts as I/O, so we need to switch our codebase or at least part of it to be async to do this in a recommended way. I have changed the signature of the `Open` method on the product source to be async. I also renamed the class to `CsvProductSource` to distinguish it from a new class we will discuss in a bit. To invoke this async method from the `ProductImporter`, the `ProductImporter` main method also has to become an async method, which is in turn invoked in an async style from our entry point. Next, I have added a new class called `HttpProductSource`. This is the class that will reach products from an internet location. This Class also implements the `IProductSource` interface, and it stores an internal

queue of products to be processed in a `Queue<Product>`. If we look at the implementation, we see that there is no closing or cleanup involved, so the `Close` method contains no code. To get a next product, we deque an item from the queue. And to see if there are more items remaining, we check if there are any products remaining on the queue. The now async open methods work by fetching the sources from a URL as a stream and then serializes that into an array of products. All the items in that array are then added to the internal queue. One thing you might notice here is that I don't provide a full URL to the file, only the last part of this URL that contains the products I want to import. I do this because the `HttpClient` allows for using a base address, which I want to control from outside of this class, for example, based on configuration or the environment like test or production that I am running this code from. To make this possible, I will have to use a preconfigured `HttpClient`. And to obtain that client, I declare it as a dependency in my constructor. And here comes a tricky thing about using an `HttpClient` as a dependency. My class requires an `HttpClient` that is tailored specifically for `HttpProductSource`, not just a general `HttpClient`, a problem that frequently occurs. Almost all users of `HttpClient` expect their client to be configured in some way, for example, with retry policies or as in this case, a base address. So let's see how we can set that up. The first thing to do is installing a new package to work with `HttpClient` from the DI container. It is called `Microsoft.Extensions.Http`. Recompiling the solution installs the package and makes the types available. This enables us to do two things. First, we inject an `HttpProductSource` as the implementing type for `IProductSource`. Secondly, we change the method we use to add `HttpClient`. Changing this method does two things. First, it registers the given type with a transient lifetime. Secondly, it provides us with a fluent interface for further configuration of the `HttpClient` that needs to be passed to the implementing type. We can now chain a new method called `ConfigureHttpClient`, which takes a lambda function that has an `HttpClient` as the parameter and allows us to preconfigure that `HttpClient` as needed by the `HttpProductSource`. In this case specifically, we set a base address like this, and I am now hard coding the value, and you should be able to figure out how to use configuration here on your own. But just in case, here's a hint. Check out the different overloads for the `ConfigureHttpClient` method, and you should know what to do. And with this change, I can now run my code and hit the breakpoint I preset in the `HttpProductSource`. Here, I can inspect the `products` variable and see that everything works as expected. There are many configuration options and choices for `HttpClient`, and we don't have the time to go over all of them. But if you want to preconfigure things like authentication, base

addresses, retry options, etc, etc, this is how to do it. Now please join me in the next lesson where we will talk about service location.

Checkpoint 09 - Register a HttpClient with a BaseAddress

Demo: Service Locator Pattern

If you have been following along closely, you have probably noticed that we have been using two different ways to get our dependencies from the container. Let's put those side by side and explore the differences. First up is dependency injection. Dependency injection is a technique where classes declare their dependencies in the constructor to ensure they get them as soon as they are constructed. This can be done either manually, as you saw at the start of this course, or using a dependency container, as you saw soon after. Another technique is the service locator pattern. Here, classes do not declare a dependency through the constructor, but they locate our dependencies on their own. If you were to draw dependency injection, it would look like this. We have classes like our `CsvProductSource` that declare outgoing dependencies on the types they need, preferably interfaces. In code, it looks like this. We have a constructor that declares the types it needs to fully understand the `CsvProductSource`. If you were to draw the service locator pattern, you would get something like this. The `CsvProductSource` would only have one dependency, namely on the service locator. Within the `CsvProductSource`, the locator is then used to find specific types the implementation needs. The service locator can be declared as an explicit dependency, but it can also be a static class. In code, it would then look like this. The `CsvProductSource` has no parameters, but instead references a static class called `Locator` to get the types it needs. This can be during construction, but also later, or you could go one step further and inject the dependency container into a type and then use it to manually resolve the types you need instead of declaring them as dependencies. We call this misusing the DI container as a service locator. Some people go so far as to declare the service locator an anti pattern. Others still believe service location has its value in certain situations. Personally, I discourage the use of service location as much as possible, and that's for two reasons. The first one is that classes that use service location are much harder to test. As they acquire their dependencies themselves, it is not as easy to replace a dependency with a mock or a stop when testing. In the next clip, you will see how easy this is when you're using dependency injection, and that's a clear benefit of DI over service

location. The other disadvantage of service location is that it hides the dependencies of a class. Instead of explicitly stating the dependencies through the constructor, it is suddenly quite unclear what the dependencies of any class are without exploring its source or documentation. Now as you saw before, injecting the container, a scope factory or a scope is also considered service location. In this course, you have seen it a few times, as working with the container directly helps with building an understanding of what it does. But whenever possible, you should avoid injecting the container directly, but use proper dependency injection where possible. Only when you have to create custom scopes, or in specific scenarios we will cover later, it makes sense to inject the container directly. Even in very large solutions, it is rarely needed to inject the container more than a handful of times. Never forget that the amount of times something is done in training does not necessarily relate to how often you will do it in the real world. Now please join me in the next clip to explore how dependency injection is a big enabler of testability.

Exercise 02 - Refactoring away from Service Locator

Demo: DI and Unit Testing

In the previous clip, you have learned that using dependency injection, either manually or using a container, is a great enabler for testing classes or part of your application in isolation. Let's explore in more detail why that is. Let's assume we find it important to test our application, responsible engineers that we are. And we want to start by testing the ProductImporter class. We take an approach where we write a unit test per requirement, and an important requirement is that we write one product to the target for every product that we read from the source. To test this from a unit test, we will have to instantiate our ProductImporter. That ProductImporter has three dependencies. So you might think we would have to instantiate those to test the importer, but to instantiate those dependencies, we will also have to install their dependencies. And you can probably guess how this story continues. Yet, if you think about it, to test this one requirement, write one product for every product read, we really only need the ProductImporter. All the other parts can either be left out or replaced with something else. And if something else, we call a mock or stop just like this. Now when your class uses dependency injection, all of this can be done quite easily. Now let's take a look at how that works. Here you can see that I have started by creating a new project called ProductImporter.Logic.Test. It's a default Visual

Studio XUnit project where I have also installed one extra library called Moq. I created a file called `ProductImporterTest`, which is going to be responsible for testing the `ProductImporter` class. Let's write our first test here together. I'm going to call this test when it reads end products, then it writes end products. This implements the test for the requirement we talked about earlier. The test will take one parameter, the number of products that we are working with during the execution of the test. Now to start our test, we will first need to instantiate the subject on the test, the `ProductImporter`. We see that there are four parameters required to instantiate this class. The first one is an implementation of `IProductSource`, and this is where the Moq library we installed comes in. Instead of instantiating a real dependency, I am going to instantiate what is called a `Mock<IProductSource>`. This Mock generates an implementation of the `IProductSource` where all the methods are just empty and at most return a default for the return type. It is thus a valid implementation, but with all the behavior removed from it. At most, it will return a default value for any returned values, so null for string, 0 for int, etc, etc. I passed this implementation that is generated by the mock as the first parameter to the constructor of `ProductImporter`, and I rinse and repeat for all the other dependencies. Now with the dependencies in place, it is time to implement the actual test. We've instantiated our subject and mocked a complete environment around it. We're now going to start the execution by invoking `subjectUnderTest.RunAsync`. And once this is completed, we verify on our product target that the `AddProduct` method has been called exactly the required number of times. And this verification is another method that comes from the mock framework. Now at this point, the test will fail for any number of products other than 0 because we need to add code that ensures it has more products, which is a method on the product source, returns true the correct number of times. There are a number of ways you can do this, and this is just one of them. If I now run the tests, you will see it succeeds. Now the test itself is not the essence of this lesson. The essence is the fact that we can easily pass in different variations of the dependencies of our subject. Due to the fact that we can use dependency injection at construction of the type to provide all the dependencies it needs, making sure that no other state around the class is required for it to work correctly, a property you will learn to value when you need to write and test increasingly larger codebases. If you want to learn more about unit testing for .NET, you can check out this course by Jason Roberts. In the course you are watching right now, we will continue by exploring how to work with optional dependencies in the next clip.

Demo: Optional Dependencies

In all our dealings with dependencies, we have never considered the case that some dependencies may be optional or not always registered. Let's change that and see how we can deal with the missing implementation of `IProductTransformer`. To simulate this, let's go into the `DIRegistrations` and comment out the registration of this type, `IProductTransformer`. Now just commenting out is not enough. We will also have to change the way we use this type in the `ProductImporter`. One approach here is to remove the injection of `IProductTransformer` and replace it with the injection of `IServiceProvider`, as I'm doing right here. The code below will now no longer compile, but we can change this by manually resolving the `IProductTransformer` from the container. At this point, we can use the difference between the `GetService` and `GetRequiredService` methods. `GetRequiredService` will throw an exception whenever it cannot find the type you are requesting just as the container throws an exception when it cannot find the dependencies of your type. But what if we use `GetService`? This method will not throw an exception, but return null instead. And we can use this value of null as a signal and use it to decide to not apply any transformations at all. This way, we have made registering an `IProductTransformer` optional, as you can see from this code running successfully. But you might be thinking, hey, this is again service locator pattern, a thing you just recommended against. Can't we do this in another way? And the answer is yes, we can do this without using service locator. So let's refer to all the changes in the `ProductImporter` class and look at another approach. Again, we go into the `DIRegistrations` file where we change the way we register the `ProductTransformer`. Instead of specifying and implementing type directly, we are now using a lambda function to create and return the implementing type ourselves. This latter has to take one parameter, which is of type `IServiceProvider`. From the body of this method, we can now return null to indicate there is no implementation. We can then again update our code to detect its null value and deal with it accordingly. However, you may have learned before that returning null from methods and trusting others to handle that is often not a good idea. So there is another strategy I would recommend, which is that of the null pattern. The essence of this pattern is that we do not return null, but instead return a full type that has no behavior, a so-called null implementation. Let's call it `NullProductTransformer` like this. You can now write a class with this name where the implementation does nothing but return the product that it has received. I believe this is something you can try out on your own, or you can find a solution in the code that is attached to this module. And as a lambda function is now no longer

required, we can return to the regular form of registering an implementing type with the service type. And that wraps up our discussion of optional types, as you have now seen three ways to deal with them. Let's meet again in the next clip to wrap this module up.

Summary

Before we move on to a new set of topics, let's take a few minutes to go over what you have learned so far. We talked about organizing our dependency registrations, first into a clear composition root and later into reusable chunks that can be consumed as a whole using extension methods. We went over some specific situations that you might want to address using the DI container, configuration using `Options<T>`, injecting `HttpClient`, and injecting an enumerable of implementing types for one service type. We talked about the service locator pattern where you inject a container directly, a thing not to do. And finally, we talked about dependency injection and unit testing. Now please join me in the next clip where we will talk about common pitfalls and some challenges you might encounter.

Common Pitfalls and Challenges

Introduction

Welcome back. In this module, we will go over some common challenges when working with the DI container in C#. We will be covering quite a few different topics, namely the following, how to deal with cases where you want to return this same implementing type from multiple service types and want to ensure it gets instantiated only once per scope or container, how hanging state can be a problem and how you can fix it, how to work with expensive dependencies that you might want to instantiate only when needed without resorting to service locator pattern, how to deal with types that need parameters to their constructor that are not known yet when the container constructs a dependency tree, and finally, how `IDisposable` types are handled by the container and what you need to think of yourself. All

right, let's get started by exploring how to instantiate only one implementing type from multiple service types in the next clip.

Demo: Implementing Multiple Service Types from One Implementing Type

In this demo, we will take a look at a scenario where we want to implement two interfaces from one class, or in other words, we want to reuse the same implementing type for multiple service types.

Opening our ProductImporter again, we can practice this by splitting the IImportStatistics in two. One thing that bugs me about the current implementation is that one interface caters for two different types of users, those that write statistics and those that read them. It is generally considered good design to tailor interfaces to the consumer, so let's do that here. I create a new interface, IGetImportStatistics and copy the one reading method to it. I also create a new interface, IWriteImportStatistics, and move the remaining methods there. I move IWriteImportStatistics to its own file and rename the original file to match the interface name. I now recompile my solution and follow the compiler errors one by one to replace all the places where I use IImportStatistics with one of the two new interfaces. And by the way, see how easy I can make a switch like this when using dependency injection. At the end, I arrive at the DIRegistrations file where I have to fix the registration of my new interfaces and remove the old one.

Lets do this in a way that we have seen before. In the existing registration, I used a new type and then copy/paste a line and register the other interface in the same way. However, if I now run the program, we do not get the desired result. No statistics are displayed at all. How can that be? The reason for this is that the container links the lifetime of an instance to the service type, not to the implementing type. In our scenario, this means that the class, IImportStatistics, is instantiated twice. One sets the implementing type for IGetStatistics and one sets the implementing type for IWriteStatistics. So let's stop the program and fix this in another way. At first, we have to ensure we only get one single instance of ImportStatistics. We can do this by registering the type with the container directly. This way, the ImportStatistics class gets its own singleton lifetime. Next, we change the registration for IWriteImportStatistics to not register the type directly, but instead use a lambda function to return the type from our own code. This lambda then takes a service provider as a parameter, which means we can resolve the ImportStatistics class here and return that instance. We rinse and repeat for the IGetImportStatistics, and we now have compiling code again or we manually redirect IWrite and

IGetImportStatistics to the ImportStatistics class. And if we run the program, we once again get the expected results. Now you know how you can register one implementing type under multiple service types, a great way to share state while retaining targeted interfaces directed at a single specific consumer. In the next clip, we will take a look at the inverse of this problem where we get shared or hanging state while we don't want it. See you there.

Exercise 03 - Implementing Multiple Service Types from one Implementing Type

Hanging State

Configuring lifetimes wrong can cause unexpected problems, for example, users that see things that don't belong to them, but to other users. It can be tricky to identify what is going wrong when this happens, so let's explore a possible case where this happens and what the cause is. Let's assume you are building a web application, for example, using ASP.NET Core that can be used for tracking things you have to do, a todo app. For getting your list of active todos, you write a TodoController that can be accessed over HTTPS. This controller has a dependency on IToDoRepository that exposes a method for getting the active todos for the authenticated user for that request, GetMyTodos. The implementation of this interface has another dependency on the ICurrentHttpContext, which can be used to retrieve all the details about the currently authenticated user. All types are registered using a scoped lifetime, so ASP.NET connects the lifetime of all the instances to the duration of a single request. All is working well. That is until the userbase of your application grows and the performance degrades. You find out that this is due to the constant invocation of GetMyTodos within the same request you want to speed things up. You do this by adding a new class called CachingTodosRepository. This repository uses the existing repository to fetch the things to do only once and then caches them for the remainder of the request. All is working well again. But your team grows and so does your userbase, so performance starts to suffer again. Another engineer sees an opportunity to speed things up even more and changes the lifetime of the CachingTodosRepository to be a singleton. They reason that this will help reduce the amount of times the database is hit and thus increase performance. However, all of a sudden, all users no longer see their own todos, but everyone sees the todos of one specific user. The reason is that the new CachingTodosRepository gets initiated

only once for the first user that requests their todos. From there onward, the same instance is reused over and over again. This is one example of a situation that can lead to what is called hanging state. If you look closely at this problem, it is actually a dependency captivity problem. The type of a singleton lifetime holds on to an instance with a lower lifetime. Dependency captivity is the most common cause of problems like this if they are due to dependency injection. Now we talked about dependency captivity before. If you want to learn more about this problem and how to prevent it, please take a look at that lesson. For now, it's time to move on to another topic, so let's talk about dependencies that you might not want to instantiate every single time, but only every now and then in the next clip.

Demo: Dealing with Dependencies That Are Expensive to Instantiate

Dependency injection provides you with some nice guarantees. All dependencies of your types are guaranteed to be provided to you at construction time. And if they're not available, you get a clear exception. But do you always want that? What if you have a dependency on a type that you rarely use, but takes quite some time to instantiate? In that case, you might be tempted to retrieve the type only when needed, for example, using service location. But service location can be a tricky thing, so let's explore if there is another way in this demo. Let's open up our ProductImporter and pretend that the ProductTransformer is a very expensive dependency to instantiate. It's also a dependency that we don't need every single time. If you're familiar with the .NET standard library, you may know of the type `Lazy<T>`, which is a default type to signal that constructing a certain type is delayed until the first use. So let's change our declaration of the `IProductTransformer` into a `Lazy<IProductTransformer>`. Once we have done this, we also need to change the way we use the variable below by first calling the `.Value` property and only then invoking the method. The value property will instantiate the `IProductTransformer` the first time it is accessed, but on further calls, it will return the previously instantiated type. Now some DI containers have built-in support for `Lazy<T>`, but the `Microsoft.Extensions.DependencyInjection` container does not. So we have to wire this up manually. We do this by opening our `DIRegistrations` and finding the registration for `IProductTransformer`. We have removed the registration for the implementing type, and instead, we use a lambda function. In this lambda function, I return a new `Lazy<IProductTransformer>`. Construction of this type also takes one parameter, which is another lambda. From this lambda, we have to return the type that we want to use when the lazy is evaluated. In this lambda body, we just have to new up the `ProductTransformer`

class and retrieve its two constructor arguments from the service provider. Now I place three breakpoints throughout my code, one here in the lambda that constructs the type, one before the call to apply transformations, and one in the constructor of ProductTransformer. Before we run the program, ensure you have disabled the automatic evaluation of properties. You can find this under Tools, Options, and then Debugging, and then General, and then the setting is right here. When I now run the program, you will see that the lambda and the constructor are not hit when the container constructs the ProductImporter. Instead, we first hit the breakpoint where we used the ProductTransformer for the first time. If I now continue, we see that the container, just-in-time, invokes our lambda to construct a type, which indeed hits the constructor as expected. Continuing from here on, we see that the breakpoint in the importer is hit two more times for the second and the third product. This time, the ProductTransformer is already available, so we do not hit the lambda or constructor anymore. Working with lambda is like this. It's an effective way of delaying instantiation of types until you really need them, but only do this when you really need to, as too much lazy invocations can make your code harder to read and harder to debug. Now in the next clip, let's explore another problem around constructors to learn how you can deal with constructor arguments that are not yet known when constructing the dependency tree.

Checkpoint 10 - Registering a Lazy Implementing Type

Demo: Constructor Arguments That Depend on Context

In the real world, not all dependencies of a class are independent of the context. Sometimes you will have to deal with a constructor parameter that is only known after constructing the dependency tree. In this demo, you will learn how to deal with those by adding an indirection through a factory. Let's open the ProductImporter again, and in this case, the ReferenceGenerator class. Let's assume we want to cater for a requirement where we want to have references that we generate to be prefixed with a certain strain, something like this. The value of this string we store in a private field for which the value is provided through the constructor, proper DI as all our dependencies are provided via the constructor. However, this isn't directly usable with DI container, as the container cannot know the value for this new construction parameter, as it depends on the context of the user of the reference generator. One way to deal with this is by adding an interaction through a factory. We first add a new

interface called `IRferenceGeneratorFactory`. This interface has one method called `CreateReferenceGenerator`, which returns a new instance of this type. The Factory method takes as a parameter all the things that depend on the context of the `ReferenceGenerator` consumer. In this case, the prefix. Next, we built an implementation. We call it the `ReferenceGeneratorFactory` and implement a corresponding interface. Here, we add a constructor that takes the dependencies of the `ReferenceGenerator`, except for those that depend on the context, the `IDateTimeProvider` and the `IIcrementingCounter`, and store them in two private fields. We can now implement the method from the interface by returning a new `ReferenceGenerator` where we combine the two private fields for which the values were injected and the one context-specific value, the prefix. And finally, to clean up the code, we move the two new types to their own files. We can now proceed and update the `ReferenceAdder` with two changes. First, we add a constant with the prefix that is used by the `ReferenceAdder`. Secondly, we take a dependency on our new factory interface from the constructor and use that in the body to build the `ReferenceGenerator` we need with the context-specific value provided. Finally, we go into the `DIRegistrations` file and update the registration from the `IRferenceGenerator` and its implementation to `IRferenceGeneratorFactory` and its implementation. And once again, we can now run the program. And if we open the results, we see that the references are updated with the requested prefixes. In the next clip, we will talk about a different topic, namely how disposable types are handled by the container.

Exercise 04 - Adding an indirection through Factory

Dependency Injection and `IDisposable`

Now let's switch gears a bit and look at another important topic around DI containers, `IDisposable`. One of the rules around classes that implement `IDisposable` is that whoever creates a disposable type is also responsible for disposing it. This rule still holds when you are working with a DI container. The container will dispose all the types that it creates and whose lifetime it manages. However, that doesn't necessarily mean you don't have to think about `IDisposable` at all. So let's explore what happens with disposable types in a few different situations. Remember this figure about the instantiation of a singleton lifetime? No matter how often you resolve it nor on what scope, you will always get the same instance returned, meaning there will be at most one instance. When it comes to `IDisposable` types,

this means that all is good as there are no large quantities of this disposable type created that hold references to all kinds of resources and even that one instance is neatly disposed as soon as the container as a whole is disposed. Now let's take a look at transient lifetimes. As you know, no matter how often or where you resolve a transient type, you will always get a new instance. When you have a type that is both transient and disposable, this means that many types might be created that are all hauling references to all kinds of resources, even resources that might be limited like network connections. Now the container will dispose of these instances when the container is disposed, but only then. Even if you create hundreds or thousands of instances throughout the runtime of your application, they will all be disposed when the container is disposed. In other words, transient types that are disposable can be an issue. They are created, but they are not disposed of until the container is disposed. Finally, there are scoped lifetimes. Here, a new instance is created for every container or scope, and this works out quite well. The container will dispose of a disposable scoped lifetime as soon as the scope is disposed, which means that there is a controlled duration for which the `IDisposable` exists. And even if this is a longer period of time, only a single instance will be created per scope. One thing that you have to be mindful of is that if you resolve a scoped disposable on the root container, it will never be disposed until you dispose the container. Depending on your scenario, this might be an issue. So what does this mean for you? What are the dos and don'ts? First of all, you should make your types disposable if that's needed. Does your class have private fields that are disposable? Implement disposable as you're used to and dispose of those fields. Next, don't register `IDisposable` types as transient to prevent them from being created, but not being disposed of when they're not needed anymore. Instead, write a factory that you can use to request the type that you need and manually dispose of it when you no longer need it. Try to avoid resolving `IDisposable` types in the root container. Even if their count might be low, their lifetime will be long, and this might have adverse effects, depending on your situation. Finally, avoid creating custom scopes with a long duration in combination with scoped `IDisposable`s. The essence here is the combination of a long-living scope and `IDisposable`. Short-living scopes like ASP.NET request scopes should pose no issues. Now let's meet in the next clip to experience some of this in practice.

Demo: Dependency Injection and `IDisposable`

In this demo, we will make a few changes to our application to explore the behavior of the DI container around disposable types. Let's open the DIRegistrations for our Transformations library and find a scoped lifetime, for example, the one of the DateTimeProvider. Opening up this class, let's make this an IDisposable type by adding the interface to the list of implemented types, generating the method, and then in the body, only adding a single line of comment and a breakpoint. This way, we can investigate when the container disposes of this type by running the debugger. When debugging, I expect this breakpoint to be hit three times, once for every product we are importing. Remember how we created the custom scope for each product we are importing? When we dispose of the scope, we should also see the disposal of the DateTimeProvider. And if you counted with me, you saw that indeed our breakpoint was hit three times. Now to prove that this was really due to the disposing of the scope and to see the impact of not disposing scopes, let's remove the using statement when we create the scope to see what happens if we don't dispose it. And what do you know? The program now runs to completion without ever hitting the breakpoint. As the scope is not disposed, neither are all the types with a linked lifetime. Let's very quickly undo this nasty change. Let's also remove this breakpoint and select another type to play with, for example, the IncrementingCounter, which is a singleton. If we again make the type disposable and add a breakpoint, let's observe what happens when we debug the program. As you can see, the first program completes by showing the statistics and only then the IncrementingCounter is disposed. This is actually due to the completion of the program, which implicitly triggered the disposal of the container and thus all singleton types. As a final experiment, let's add a new class called the DisposingProductTransformation, a new implementation of the IProductTransformation that is also disposable. We don't add any behavior to this type, but only place a breakpoint in its Dispose method. To wire things up, we register it as a new implementation of the interface and give it a transient lifetime. If we now run the program, we will see that the type is disposed three times, as it is instantiated three times in three different scopes. But if we inspect the call stack very closely, we see that it is disposed from the end of the applied transformation methods. This is due to the fact that this is the method where the scope is instantiated. At the end of the code block where that is done, the scope is automatically disposed due to the using statement. Here, we see that transient types can outlive their usage if the scope or container they are instantiated by is long-lived, a thing to be wary of. This demo has showed you in practice some of the details around disposable types when using a DI container. Now let's meet in the next clip to wrap this module up.

Summary

Once again, we are at the end of a module, so it's time to summarize what you have learned. In this module, we talked about some common challenges when working with the Microsoft DI container. We discussed how you can register the same implementing type for more than one service type under a single lifetime. We talked about hanging state, problems it can cause, and how it is often caused by dependency captivity. From there on, we talked about construction-related challenges. First about how to defer the construction of a type until the last possible moment, especially helpful in cases where you are working with dependencies that are not used in every code path. We also talked about how to deal with constructor parameters that depend on the context of the consumer and how you can supply those by indirecting through a factory. Finally, we talked about `IDisposable` and how to work with disposable types using the DI container. I hope to see you in the next module where we will talk more about advanced techniques that you might need only in specific situations, but are great to have in your toolbox.

Advanced Techniques

Summary

In this final module, you will learn about DI techniques that you might use less frequently, but are good to know about so you can apply them in those very specific situations where you might need them. We will first talk about using the DI container for A/B testing, a technique where you have two variations on the code path in your application and want some users to use the first one and other users to use the second one. You will learn how to switch away from the default Microsoft container to another container, in this case, Autofac. Once that is done, we will take a look at a few Autofac techniques that are not supported by the default Microsoft container, namely property injection, built-in support for `Lazy<T>`, interception, and assembly scanning, but let's start with some A/B testing.

Demo: A/B Testing

Changing existing software is a risky thing. While box and new functionality are, of course, undesirable, box in existing functionality, the so-called regressions are often worse. Instead of delaying something new for a user, you are breaking something that is already there. A/B testing is a technique where you not just change something in your application, but keep the existing behavior for a while and add a new behavior next to it. You then at runtime decide which behavior to execute. You can do this based on configuration, the logged-in user, the previous action or whatever makes sense in your scenario. If you want to implement such a thing, you can do that as follows. You have your consuming class that takes a dependency on an interface. But now instead of having one implementation, you have two. And at runtime, you switch between the two of them. In many cases, implementation B is not completely new, but a copy of implementation A that receives the new changes. This technique is also called branching by abstraction and is used for risk reduction when changing existing behaviors. Let's give this a try in our ProductImporter and use the DI container to do it. Let's assume you want to change the CsvProductTarget, for example, to change its implementation for writing CSVs to a more polished approach using the CSV helper package. But you want to make this change in such a way that you can enable it per user through configuration. Let's do that together. The first thing I do here is that I copy the CsvProductTarget class to another file and call it OldProductTarget class, not the best name, but it serves our purpose. The goal is to remove this class in a few days or at most a few weeks once our new implementation is battle tested and all users are migrated to this new class. I also update my DI configuration to use the old class, and running the application proves that everything is still working. But I now have a CsvProductTarget class that I can change, and test, and ship to production without ever impacting a single user. So let's start making changes. First, let's install the CsvHelper NuGet package we wanted to use. I can now update my class to hold on to a CsvWriter instead of a StreamWriter. I need to instantiate the CsvWriter, which will actually wrap a StreamWriter. So let's make this a local variable and use that to instantiate the CsvWriter. I also changed the code for writing the header to the CsvWriter's approach. And observe how I do not have to use our own specific _productFormatter anymore, but can rely on the CsvWriter to figure the headings out on its own. Moving further down, I update the null guard and change the way a record is written. I omit our own custom formatting and omit a product directly, and don't forget to leave the IncrementOutputCount in if you're following along. Finally, I update the Close method. At this point, we can observe that we are no longer using the _productFormatter, so I am removing that

from our code. I run a compile to check if everything is in order, and it is. Finally, as the `CsvWriter` is disposable, I also mark my `CsvProductTarget` as disposable and dispose of the `CsvWriter` in the `Dispose` method, and my class is ready to go. It is now time to update the `DIRegistrations` to start using this new class. At first, I upgrade the registrations for the `IProductTarget` from type to type to using a lambda function. In real-world cases, you can base this off the logged-in user, the day of the week or the date if you want to include an Easter egg, for example. In this case, I'm going to change based on the configuration. So at first, I resolved a class that holds the configuration. Then in an if class, I get the value for `EnableCsvWriter` as an integer. And if that equals 1, I am going to return my new implementation. Please note that this is valid, as DI configuration will not error on missing values, but instead return the default of int, which is 0. This code will thus work in all possible cases, even if the setting is not there. To finalize this change, I have to register both the `CsvProductTarget` and the old variation with the container so that I get required service calls in my lambda function work. Now we are ready to test our work. I do this by selling breakpoints in the constructors of both classes that implement the interface and run the application. As there is no setting in our application configuration yet, I expect the program to hit the breakpoint in the old class. And as you can see, that is exactly what it does. I now let the program run to completion and update my `appsettings.json` to include this new setting. Set its value to 1 and run the program again. This time, we will see that the new code is being hit. Conclusion, everything works as expected. If you haven't been using branching by abstraction or A/B testing yet, I hope that this clip inspires you to give it a try to gradually expose your code changes to your customers as well. Now please join me in the next clip to talk about switching containers.

Demo: Choosing Another Container

At the start of this course, we briefly talked about the different containers that are available when working with C#. In many cases, you can stick with the default Microsoft container, but in some cases, you might want to choose another container. So what might those cases be? First of all, they can come from the past. If you have an existing codebase with thousands or millions of lines of code and everything is working with another container, you might not be inclined to refactor everything to use the Microsoft container instead. While your code should aim to be independent of the container that is supporting it, in practice you might also have some unexpected dependencies on very specific implementation details or simply don't want to take the risk. A second reason might be that another

container has specific capabilities you need. The Microsoft default container has quite a lot of features that are good enough for building standard applications, but your application might require something more special, and another container has that capability. Finally, a third argument that you might hear is that of performance. However, recent benchmarks show that the Microsoft container has a pretty decent performance and that there are only minimal gains from moving to another container. Only if your application has very high performance needs, this might be a good argument. In that case, be very particular about which container you pick. Two common containers for .NET are Autofac, which is very mature and feature-complete, and Ninject, which focuses on simplicity. Autofac is very popular. It has hundreds of millions of downloads on NuGet. So you wouldn't be the first user, and it is unlikely you are taking a dependency on something that is going away any time soon. It is very feature-complete and also supports a number of more niche capabilities of which we will explore a few in the following clips. One feature that was very valuable in the past was the built-in support for modules as a way of organizing your registrations into groups, the thing we did manually for the Microsoft container. Ninject is also very popular, but has fewer downloads than Autofac. It doesn't have as much features, as it is focusing on simplicity. If you are ever in a scenario where you can't use the Microsoft container, but don't need an advanced feature either, Ninject might be a good choice. Ninject is also lightweight and portable, meaning it also runs on the open-source implementation of .NET called Mono. In the next clip, we will switch to Autofac in the ProductImporter so we can explore a few of its capabilities later. See you there.

Demo: Switching to autofac

Now you know about two other containers that you might use instead of the default one. Let's implement Autofac in the ProductImporter. To start with this, we have to install the correct NuGet package into three of our projects. We do this by right-clicking the Solution and choosing Manage Nuget Packages. The package I need to install is called Autofac.Extensions.DependencyInjection and should be installed into the main Project, the CompositionRoot, and the Transformations project. Once installed, we go back to the Program.cs file and remove the configuration for the default container and replace it with a call into UseServiceProviderFactory, which allows us to select another service provider, a.k.a, DI container. We pass a new AutofacServiceProviderFactory as the value, and we have now effectively replaced the Microsoft container with Autofac. The existing DIRegistrations,

however, will continue to work. Don't believe me? Let's add a throw into a random constructor and press F5 to see what happens. As soon as the exception opens, I view its details. Go to the stack trace, and there you have it. Autofac is controlling the instantiations. Now let's quickly remove these throws and go back to the main program. Let's assume we also want to change some of our registrations to use Autofac-specific features. To make this possible, we chain another call onto the host builder called `ConfigureContainer`. This is a generic method that you can use to configure the actual DI container, similar to `ConfigureServices`. You will have to specify the type of the container you want to use though, which for Autofac is `ContainerBuilder`. This method takes a lambda as a parameter, which allows us to start configuring that container. Here, we can directly register types with Autofac using the Autofac syntax, `containerBuilder.RegisterType`, which takes the implementing type as a generic and then `.As`, which takes the service type as a parameter, but let's not. Let's extend our current pattern of using a composition root and invoke an extension of it on the `ContainerBuilder` with the same name as we do for the Microsoft container. To make that work, we have to create that method somewhere, and the `CompositionRoot` is a good place. I add the extension method. And switching back to `Program.cs`, you can see this now compiles. By the way, this is a good point to make an observation. Yes, you are seeing this correctly. I am going to mix registrations using the Microsoft and the Autofac approach, and this is perfectly valid. The registrations using the Microsoft methods are under the hood relayed to the Autofac container. And once complete, my Autofac registrations are invoked as well. This approach is a great way to migrate from one container to the other. Continuing the implementation, let's not directly add registrations here, but continue with our pattern of pushing registrations down, down to the site next to the times that it registers. Let's do this for the transformations, which are currently registered on lines 13 to 15. Autofac has a built-in pattern for this called modules. So we invoked a method, `RegisterModule`, and passed in the type that holds the registrations, which in our case will be called `ProductTransformationsModule`. As we own the constructor, we can also pass along extra values like the `ProductTransformationOptions` that we use to enable or disable certain functionality. This is equivalent to the lambda we pass along on line 14. Finally, let's remove the Microsoft-style registrations, as we are building something to replace them. Now let's rename the `DIRegistrations` class for our transformations and rename that to `ProductTransformationsModule`. We open up the file and make this class non static so we can new it up and then have it implement `Module` from the Autofac namespace. We add the constructor to accept

the `ProductTransformationOptions` and store them in a private field. We then add a new method, which is an override of a method in the module type called `Load`. This method receives the `ContainerBuilder` again, so this is where we add the actual registrations, and we can move them over from the old function below one by one. I call into `RegisterType` and specify the implementing type I am registering followed by a call into `As`, specifying service type. This is the opposite order as it is with the Microsoft container, which can be a little bit confusing. Finally, I specified a lifetime. Autofac has up to six different lifetimes, including three that are completely equivalent to what we have been using so far. And this is a scoped lifetime, so I add a call into `InstancePerLifetimeScope`. From here it is rinse and repeat, so let me speed it up a little bit. You can follow along or get the code changes from the assets with this module. Now the third registration we have to do is a transient one. Transient is the default lifetime in Autofac, so I can leave the call to the lifetime completely out if I want. But I like to be explicit, so I invoke `InstancePerDependency`, which is equivalent to transient. I then speed things up again until I arrive at a singleton lifetime. Here, the Autofac equivalent is a call into `SingleInstance`. And after a final minute of coding, all registrations are complete. I can compile and run the application, and voila! Everything runs as it did before, but now using the Autofac container. Now let's meet in the next clip to see how Autofac can help us deal with optional dependencies.

Checkpoint 11 - Switching to Autofac

Demo: Optional and Missing Dependencies

When talking about the default Microsoft container, we also talked about dealing with dependencies that are not frequently used and for which you want to delay the instantiation until you really need the type. We also talked about dealing with optional dependencies using the null pattern. Now let's revisit these two topics and see how Autofac can help us deal with `Lazy<T>` constructive parameters that have a default value of null and property injection. Let's start by exploring `Lazy<T>` and how that is handled by Autofac. I have opened up one of our classes, the `ReferenceAdder`, and I have decided that I want to inject the `referenceGeneratorFactory` as a lazy instead of directly. Switching to the module with the registration, I can see that this type is not registered as lazy, but only directly. Now that's not a problem, as Autofac supports `Lazy<T>` out of the box. Let's verify this by changing our usage to be lazy, updating the code to still compile, and then adding a breakpoint. Running the

application now shows that the value is injected as a lazy out of the box, a handy thing supported by Autofac without any extra work. This way, we can delay the construction of types without manually messing with DIRegistrations, or we can even forgo initiating it if we do not need the dependency in a specific code path. Now let's take a look at optional dependencies that may or may not have an implementing type registered. We can simulate this by injecting a `Lazy<IHaveNoImplementation>` and generating that interface. If I now run the code again, we will see an exception explaining that the reference error cannot be constructed by Autofac due to the fact that no constructor can be found for its Autofac can provide all dependencies. So we need to try something else. A thing we can do here is inject the type directly, but give it a default value of null. If I now run the program again, you will see it runs without exceptions. The constructor is invoked and simply no value is specified for `IHaveNoImplementation`. Our code can now use the null value for this dependency as a signal that it is not specified and should work without it. Another way this can be done is by using property injection. When using this technique, you do not declare dependencies through the constructor and have a backing private field, but instead specify dependencies as public properties with a getter and a setter. These are also the changes you see me making here. I also removed the private field and the parameters from the constructor. And to make everything compile, I have to change the usage of the private field to the usage of the public property. I also set an extra breakpoint at the start of the `Execute` method, so we can inspect what happens when we run the application. But before I can do that, we have to make one final change to the `DIRegistration`. We need to chain a call to `properties` out wired to the registration of the reference error. This instructs Autofac to also scan the time for public properties and set values for those if they can be resolved by the container. Now let's run the application to see what happens. When I first break in the constructor, you can see that the value for both properties is still null. Yet, if I continue, you can see that when entering the first method, the second property is no longer null. Autofac has scanned all properties and set a value from the container for those it could resolve. This is another way for dealing with optional dependencies, as this approach will not fill all missing types, but just continue with the next property. The last two techniques you saw, a default value of null and property injection, are not patterns I would recommend to use by default. Yes, they work, but they are not as obvious and intent-bearing as plain and simple constructor injection in my opinion. Use this only when there is no other option. That being said, let's explore another powerful Autofac feature called interception in the next clip.

Exercise 05 - Working with property injection

Interception

Another powerful feature of Autofac is interception. Let's explore together what that is and why you might want to use it. Interception is a practice where you wrap classes called interceptors around an implementing type. This means that all calls to the implementing type first go to the interceptor or interceptors and then to the implementing type and return the same way. If you are familiar with the decorator pattern, interceptors are a way for implementing decoration. If you're not familiar with decorating, this might sound very abstract, so a diagram might help. As you have seen many times by now, we start by having a class that consumes an interface by taking a dependency on it. We then add another class that implements this interface. If we wire this up using a DI container, this means that at runtime, the consuming class can make calls to the implementing type. Now when you add interception to an implementation, the container will generate an implementation of the interface. This generated implementation will implement all the methods in the interface by calling into the first interceptor that is registered on the service type and then go through all the interceptors one by one and finally go into the implementation. Once the implementation returns a void or a result, control returns to the last interceptor all the way up to the consuming class. But why would you want to do this? Of course, there are many use cases, depending on the application, but a common one is logging. Let's say you become responsible for operating an application without any logging and no time to add detailed logging to it. In that case, you could instead apply an interceptor to all implementing types and let the interceptor log the calls. Not the best logging implementation, but definitely a quick one. Another example might be when you find out that your application is leaking information about its internals in exception messages that bubble up to the callers of an external interface. In that case, you might want to catch and log all these exceptions and throw a new exception that doesn't reveal the original message. Now let's explore together how to implement call logging in our ProductImporter in the next clip.

Demo: Interception

In this clip, you will learn how to add interception using Autofac, in this case, to automatically log all calls to our methods in the ProductTransformation project. To start using interception, we first have to

install another NuGet package called `Autofac.Extras.DynamicProxy`. This is the package that can be used for generating the implementation that calls the interceptor for every interface method. Once this is done, we can add a new class that will be our interceptor. Let's call it `SystemOutLogger` and make this a public class. From this class, we will have to implement the `IInterceptor` interface, which is used from the generated implementation. Upon implementing this interface, you see it has only one method. This method takes one parameter, and that one parameter contains all the information about the method that is being called. Here, you can execute your own code, but it is important to also call `Invocation.Proceed`. This method will call the next interceptor or if this is the last interceptor in the chain, the actual implementation. With this call in place, we can now add our own custom code, and I create three local variables that I populate with some properties of the call we are intercepting, the full name of the target type that is being called, the name of the method that is being called, and all the arguments that are being supplied, identifying all these things into a single message about the method that is being invoked with the parameters. Next, I move the cursor on to after the invocation where I log that the call is now completed. And now it is time to register the interceptor with the container in our `ProductImporterTransformationsModule`. First, we register the interceptor itself by calling `builder.Register` and providing it with a lambda function. We don't use its parameters and directly return a new `SystemOutLogger`. We also have to specify which types we want to intercept, and we do this by first enabling interception by chaining a call into `EnableInterfaceInterceptors` and then calling `InterceptedBy` to which we have to provide the type of our interceptor. These two calls we then copy to each and every registration. If I now run the program, we see that all the calls to all the types in this module are intercepted and logged, precisely what we wanted. As you can see, interception is a powerful tool, and you can probably find much better uses for it. But for us, it's time to stop here. Let's meet in the next clip to look at one final Autofac feature called assembly scanning.

Exercise 06 - Adding interception

Demo: Assembly Scanning

In this final demo, I want to show you one Autofac trick that can be handy when you want to create many registrations at once. You can use this, for example, when you have an existing application without a DI container and want to start adding one. If we open up our Autofac module full of

registrations and scroll through it, we can count a total of five registrations of an `IProductTransformation`, all registered with the same lifetime, all registered with the same interceptor. Can this be done easier? And well, yes, it can. So let's go through our code and remove all these registrations. Instead, I will invoke another method on the builder, namely `RegisterAssemblyTypes`. This is a method that allows us to select all the types in a specific assembly for registration. In this case, I can get the type of my current class and use that to find the assembly that contains it. I can now use one or more LINQ queries to trim the selected set of types down again until I have those that I really need. Filter that is very handy in particular is the `IsAssignableTo` method. It allows me to filter down to all the types that are an implementation of `IProductTransformation` or more precise can be assigned to a variable of type `IProductTransformation` due to polymorphism. With the set of types selected, I can now change the regular set of methods you have seen before, as they will automatically be applied to all the types in the selection. This is also the reason I do not use the `As` method you saw before, but I call `AsImplementedInterfaces` instead. This will automatically register every type under the interface or interfaces it implements. I then add the other method calls we had before, and my solution is ready to go. If I now run the program one final time, we can see everything still runs as expected without any mistakes. Our logging interceptor proves that all product transformations are invoked one by one, and I am one satisfied user of this final version of the `ProductImporter`. Now registering types like this is not necessarily something I would recommend as a default approach. Selecting many types at once based on a selector might also include types you don't want to register, but are created without you knowing it, possibly after you created the query. Yet if you have to wire up a large solution using DI and you want to make some quick steps, assembly scanning might be the tool for you. It is also the final thing we will discuss in this course, so please join me in the next clip to recap what you have learned.

Checkpoint 12 - Using assembly scanning

Summary

Before we wrap this course up, let's review what you have learned in this final module. First, we talked about A/B testing, an approach to gradually release new features using branching by abstraction. Next, we talked about how to switch from the default Microsoft container over to Autofac and explored

some Autofac-specific features. You saw how to use property injection and Lazy<T> to deal with non frequently used or optional dependencies. You also learned how to use interception to quickly add functionality to a group of classes and finally how you can take a shortcut to registering types using assembly scanning, advanced techniques you might not use every day, but just might save the day sometime in the future. To wrap this course up, let's meet in the next clip to take a look at what you can learn next.

Thank You

Now that you have gone through this course, you might be wondering what's next. There is so much to learn on Pluralsight, but there are two things that I want to call your attention to. The first one is the course, Dependency Injection in ASP.NET Core by Steve Gordon, a logical next step if you are working on ASP.NET applications and want to learn how to apply all you have learned about DI in that specific framework. The second one is the C# learning path. There is so much more to learn about working with C#, and I want to encourage you to explore this path to find more courses that fit your needs. But no matter how you continue learning, congratulations on completing this course, a great feat. I hope you enjoyed this course and will rate it or leave a comment. Thank you for your time, and hopefully until the next time.